Received: Added at production | Revised: Added at production | Accepted: Added at production

DOI: xxx/xxxx

EXPERIENCE REPORT

Use the model, ship the program: neuro-guided synthesis of very fast learners

Tim Menzies

[1]Computer Science, North Carolina State University, NC, USA

Correspondence
Tim Menzies. Email: timm@ieee.org

Summary

On [N] tabular SE data sets, a Naive Bayes classifier over seven discretized bins comes within [X]% of LightGBM and TabICL (by recall and false alarm). The whole learner is [N] lines of Lua with no dependencies; it trains and tests in [T] seconds on a laptop. We describe its design, the experience of keeping it small, and the working style it enables: many fast runs, and only an occasional slow check against an expensive reference.

KEYWORDS:
naive Bayes, discretization, baselines, Lua, little languages, reproducibility

1 | INTRODUCTION

MSR May 19, 2013

Every month brings a bigger model. Every month, last month's results look old.

This paper offers a result that will not age so fast. On [N] tabular SE data sets, a Naive Bayes classifier with seven discretized bins comes within [X]% of gradient boosting (GBM, LightGBM) and TabICL, measured by recall (pd) and false alarms (pf). The whole learner is [N] lines of Lua. It runs in [T] seconds on a laptop. A student can read it in an afternoon.

After two years of intensive work with LLMs, writing papers on software analytics, we assert that science-oriented programming is different from commercial programming. Commercial code optimizes for delivery: ship the feature, minimize time and cost, and let size and opacity hide behind test suites and teams. Scientific code optimizes for something else – the veracity of a conclusion. Here, the program is not a product that supports the argument; the program *is* the argument, and every line is a claim that a stranger must be able to check.

That difference makes minimality an end in itself, not an aesthetic. A scientific deliverable is judged by how quickly a skeptical reader reaches either "I believe this" or "here is your bug." So we practice *seeking minima as a methodology*: delete until nothing can hide. Seeking minima is not simple, since a minimum means little until it is benchmarked against the maximal alternative – hence every small learner here is scored against LightGBM and TabICL, so the cost and the benefit of smallness are both on the table. The reward is what the minima teach: each dramatic simplification found below (one update primitive, training inside constructors, learners as two or three questions asked of shared summaries) was a thing we did not know until the surrounding code was gone.

There is also a lesson here about building neuro-symbolic systems. This kit was written with an LLM used as a sounding board, a tool for exploring options, a research assistant doing the mechanical work – while a small symbolic artifact (a few hundred lines of Lua, pinned by golden tests) disciplined every suggestion: any change had to reproduce byte-identical output and hold the baseline speed, or it died. Many fast probes, aggressive deletion, and an executable referee between the two. We offer that loop as one answer to how to do science in the age of LLMs.

We do not claim that big tools are useless. We claim that most of the time, they are not needed. Science works best with many fast experiments and only a few slow ones. So we should run many cheap probes and check against an expensive model only now and then.

This is an old idea. In the 1960s, information systems buried managers in reports. Decision support systems replied with a question: what is the least information that most changes a decision? AI now buries us in code. We ask that question again, and we answer it in a small language.

We wrote this for three readers:

- Scientists who want results they can repeat on a laptop, and that will still hold after the next model release.
- Teachers who need code students can read end to end. Schrödinger warned that if you cannot, in the long run, tell everyone what you have been doing, "your doing has been worthless"[1]. Small code makes that telling possible.
- Lua users who already believe a little language, used well, goes a long way[2]. Here is evidence.

Our contributions are:

- An empirical result: a 7-bin Naive Bayes is close to state-of-the-art on pd/pf for [N] data sets.
- A method: many fast runs, one slow check.
- A tool: tab.lua, [N] lines, no dependencies.
- An argument: some results outlast the current model cycle, and we should look for them.

2 | MOTIVATION

2.1 | Overload, again

In 1938, Buckminster Fuller described ephemeralization: doing "more and more with less and less"[3]. Technology delivered that promise, then delivered too much. Gross gave the problem a name in 1964[4]. Toffler made it famous in 1970[5]. In between, Ackoff saw it inside companies. Managers did not lack relevant information. They drowned in irrelevant information[6]. Simon put it in one line: "a wealth of information creates a poverty of attention"[7].

The answer then was decision support[8]. Do not give people every report. Give them the smallest thing that changes what they do next.

LLMs repeat the cycle. Code and text now cost almost nothing to produce. Reading them, checking them, and trusting them still cost a lot. So the problem has moved from making things to choosing among them, and the right response is the old one: find the minimum that matters.

2.2 | Programming under examination

Repetition matters because we all make mistakes. In 2013 we had a paper accepted at MSR on cross-company defect prediction. It beat the prior state of the art. Within three weeks, before the conference started, a graduate student trying to reproduce it found a coding error: a bad re-insert of class labels after a distance calculation. Once we fixed it, the result went away. We reported this at the conference. That error was found because the code was small enough to read, and the data was open enough to rerun. Large, opaque, expensive pipelines hide errors like this. Small, cheap ones expose them.

That episode set a working rule we have kept since: offer everything. All the code, all the data, the seeds, the splits, the scripts that drew every figure – so that others can repeat, improve, or refute the result at the cost of one download. This is partly service and partly discipline. Write every line expecting that it will be examined, in great detail, by some researcher somewhere in the world, and the line comes out different: shorter, plainer, with its assumptions on the surface. Reputation in science is not built on being right the first time; it is built on being checkable, and on reporting quickly when the check fails.

Big models work against this. A result that needs a GPU cluster can be checked only by people who have one. Most of the world's scientists, students, and small labs do not. For them, current AI research is something they read about but cannot repeat. When results cannot be repeated by most people, it is hard to call them science.

2.3 | What the logs say

The difference can be measured. This kit was built with an LLM assistant, and the assistant's transcripts record every request made of it. Over two months and 868 prompts (this kit, plus four sibling research projects), the requests sort as follows: 105 asked for renames or uniformity ("is `i` only ever used for self?", "constructors before methods"); 72 interrogated code that already worked ("why do we need `leaves`?", "surely only `Num` needs `w`?"); 61 demanded verification (golden tests, byte-identical output, "same job at same speed as the lua we started with"); 57 asked for something to be deleted, folded, or shrunk. Not one asked for faster delivery. (The classifier is `etc/logmine.py` in this paper's repository: thirty lines that read any Claude Code transcript directory, so readers can audit our audit – or their own practice.) A commercial reviewer would read most of that traffic as churn. Here it is the work: the majority of the editing budget went to making the artifact smaller, more uniform, and more interrogated, under a regime where behavior and speed were frozen by tests. Form is content when the artifact is the argument.

2.4 | The ground keeps moving

There is a third problem. Whatever we build on today's model, the next model will replace. A paper tuned to one release has a short shelf life. Researchers end up on a treadmill, spending effort to keep up instead of to understand.

So we look for results that do not depend on the current release. Some ideas have lasted decades: discretize, count, compare, keep the simplest model that works. A 7-bin Naive Bayes will mean the same thing in ten years as it does today. Its baseline does not expire. When a new model arrives, the right question is still "how much better than the simple thing?" And often the honest answer is "not much."

2.5 | Little languages, fast runs

Our tool choice follows from this. Lua is a small language with a small core and few dependencies[2]. It changes slowly, so code written years ago still runs. It encourages short scripts that do one thing.

That suits a research style with two speeds:

- Fast: thousands of cheap runs with small learners to explore the option space.
- Slow: a few runs against an expensive reference (GBM, LightGBM, TabICL) to confirm what the fast runs found.

Most of the time goes into the fast loop. The slow loop is a check, not the main tool.

3 | COLUMNS

Columns summarize streams: counts for symbols, Welford's running mean and standard deviation for numbers.

Listing 1: `Sym`

```
function Sym(txt,at) --> SYM; a SYM for ‘txt:str‘ at ‘at
    ↪ :num‘: a-z names,
  return new(SYM, {txt = txt or "S", at = at or 0, n =
    ↪ 0, has = {}}) end
```

Listing 2: `SYM.add`

```
function SYM.add(i,v,inc) --> value; add one more ‘v:
    ↪ value‘ to ‘i‘’s counts.
  if v == "?" then return v end
  inc = inc or 1
  i.n = i.n + inc
  i.has[v] = (i.has[v] or 0) + inc
  if i.has[v] <= 0 then i.has[v] = nil end
  return v end
```

Listing 3: `Num`

```
function Num(txt,at) --> NUM; a NUM for ‘txt:str‘ at ‘at
    ↪ :num‘: A-Z num, !+-
  return new(NUM, {txt = txt or "N", at = at or 0, n =
    ↪ 0, mu = 0, m2 = 0,
                   w = (txt or ""):find"%-$" and 0 or
    ↪ 1}) end
```

Listing 4: `NUM.add`

```
function NUM.add(i,v,inc,    d) --> value; add ‘v:value‘
    ↪  to ‘i‘, returning
  if v == "?" then return v end
  inc = inc or 1
  i.n = i.n + inc
  if i.n <= 0 then i.n, i.mu, i.m2 = 0, 0, 0; return v
    ↪ end
  d    = v - i.mu
  i.mu = i.mu + inc * d / i.n
  i.m2 = i.m2 + inc * d * (v - i.mu)
  return v end
```

4 | NAIVE BAYES

Training is the constructor: one sub-table per klass.

Listing 5: `Nb`

```
function Nb(tbl,rows,    i,col,v) --> NB; one sub-TAB
    ↪ per klass in ‘rows‘
  col = tbl.klass
  i   = new(NB, {tbl = tbl, subs = {}, n = #rows, nKlass
    ↪  = 0})
  for _, row in order(rows) do
    v = row[col.at]
    if not i.subs[v] then
      i.subs[v], i.nKlass = tbl:clone(), i.nKlass + 1
    ↪ end
    i.subs[v]:addRow(row) end
  return i end
```

Listing 6: `NB.like`

```
function NB.like(i,sub,row,    fn) --> num; log P(‘row‘
    ↪ | ‘sub:TAB‘)
  fn = function(col1,at,    col,v)
       col, v = i.tbl.x[at], row[col1.at] -- col knows
    ↪ every value;
       if v == "?" then return 0 end      -- col1 knows
    ↪  this klass's
       return log(((col1.has[v] or 0) + the.m)    --
    ↪ and m per value
                  / (col1.n + the.m * col:nVals())) end
  return log((#sub.rows + the.m) / (i.n + the.m * i.
    ↪ nKlass)) -- m/klass
       + sum(sub.x, fn) end
```

Listing 7: `NB.guess`

```
function NB.guess(i,row,    fn) --> value; ‘row‘’s
    ↪ likeliest klass.
  fn = function(sub) return i:like(sub, row) end
  return (best(i.subs, fn)) end
```

5 | RANGE MEANS

Distance to heaven labels a row; bins collect those labels; a guess is the variance-weighted, shrunken mean of the bins a row falls into.

Listing 8: `TAB.ydist`

```
function TAB.ydist(i,row,    fn) --> num; ‘row‘’s
    ↪ distance to heaven over
  fn = function(col)
         return abs(col:norm(row[col.at]) - col.w) ^
    ↪ the.p end
  return (sum(i.y, fn) / #i.y) ^ (1 / the.p) end
```

Listing 9: `Rmeans`

```
function Rmeans(tbl,rows,y, --> RMEANS; per x column,
    ↪ its bins' y means
                sub,all,i,bins,b,w,fn)  -- and the
    ↪ variance they cut
  sub = tbl:clone(rows)
  all = nums(sub.rows, y)
  i   = new(RMEANS, {all = all, has = {}, cols = sub.x})
  for _, col in order(sub.x) do
    bins = {}
    for _, row in order(sub.rows) do
      b = col:bin(row[col.at])
      bins[b] = bins[b] or Num()
      bins[b]:add(y(row)) end
    fn = function(v) return v.n/all.n * v:div() ^ 2 end
    w  = all:div() ^ 2 - sum(bins, fn)
    i.has[col.at] = {bins = bins, w = max(0, w)} end
  return i end
```

Listing 10: `RMEANS.guess`

```
function RMEANS.guess(i,row,    s,w,it,v,mu) --> num; '
    ↪ row''s guess: bin
  s, w, mu = 0, 0, i.all.mu
  for _, col in order(i.cols) do
    it = i.has[col.at]
    v  = it and it.bins[col:bin(row[col.at])]
    if v and it.w > 0 then
      s = s + it.w * (v.n * v.mu + the.m * mu) / (v.n +
    ↪ the.m)
      w = w + it.w end end
  if w > 0 then return s / w end
  return i.all.mu end
```

References

1. Schrödinger E. *Science and Humanism*. Cambridge University Press . 1951.
2. Ierusalimschy R, Figueiredo dLH, Celes W. Lua—an extensible extension language. *Software: Practice and Experience* 1996; 26(6): 635–652.
3. Fuller RB. *Nine Chains to the Moon*. J. B. Lippincott . 1938.
4. Gross BM. *The Managing of Organizations*. Free Press of Glencoe . 1964.
5. Toffler A. *Future Shock*. Random House . 1970.
6. Ackoff RL. Management misinformation systems. *Management Science* 1967; 14(4): B147–B156.
7. Simon HA. Designing organizations for an information-rich world. In: Johns Hopkins Press. 1971 (pp. 37–72).
8. Gorry GA, Scott Morton MS. A framework for management information systems. *Sloan Management Review* 1971; 13(1): 55–70.
9. Menzies T. tab: fast inference over tables. 2026.

